

DataPrep AI Copilot

Guide Frontend

Rôle, Design & Architecture de l'interface utilisateur

Version 1.0 — Avril 2026

1. Résumé du projet

DataPrep AI Copilot est une application web qui automatise l'exploration et le prétraitement des données tabulaires (CSV, Excel). Le système guide l'utilisateur à travers une boucle intelligente : analyse statistique du dataset, détection des problèmes de qualité, proposition de solutions expliquées par un LLM, application des transformations validées par l'utilisateur, puis re-analyse jusqu'à obtenir un dataset propre et prêt pour le machine learning.

L'application est pensée comme un copilote pédagogique : l'utilisateur comprend et valide chaque transformation avant qu'elle ne soit appliquée. Il n'y a pas de boîte noire — chaque décision est expliquée, argumentée et réversible.

Dimension	Description
Utilisateur cible	Data scientists, analystes, développeurs ML qui veulent préparer un dataset rapidement sans écrire de code à la main
Format données	CSV et Excel (.xlsx, .xls) jusqu'à 50 MB
Backend	FastAPI + PostgreSQL + Gemini (LLM)
Session	Anonyme par UUID, sans authentification, persisté en localStorage
Déploiement	Frontend sur Netlify, Backend sur Railway

2. Philosophie de l'interface

Le frontend n'est pas un tableau de bord passif. C'est une interface de dialogue active entre l'utilisateur et le système. Trois principes guident sa conception :

- **Progressivité** : l'utilisateur n'est jamais submergé. Les informations apparaissent étape par étape, dans l'ordre logique du flux de prétraitement.
- **Pédagogie** : chaque recommandation est expliquée avec le raisonnement du LLM visible, pas seulement le résultat.
- **Contrôle** : l'utilisateur choisit, confirme, peut rejeter une suggestion et demander une alternative.

Le frontend est un composant de premier ordre de l'application — pas un add-on. La qualité de l'expérience utilisateur détermine directement la valeur perçue du produit.

3. Architecture des pages

L'application est organisée en 4 pages principales avec un flux de navigation linéaire mais non bloquant — l'utilisateur peut revenir en arrière à tout moment.

Home → EDA → Diagnostic & Preprocessing → Export

3.1 Home — Upload & Configuration

Point d'entrée de l'application. L'utilisateur dépose son fichier et configure le contexte de son analyse.

Composants à afficher

- Zone de drag-and-drop pour l'upload CSV / Excel, avec indicateur de taille et format accepté
- Champ de sélection de la colonne cible (liste déroulante peuplée après upload)
- Sélecteur du type de tâche : Classification / Régression / Exploration
- Champ texte libre pour l'objectif de l'analyse (contexte transmis au LLM)
- Bouton de validation qui lance l'analyse complète

Comportement attendu

- À la soumission : appel POST /api/datasets/upload puis POST /api/analyze/{id}
- Pendant le chargement : skeleton loader ou spinner avec message d'attente
- En cas d'erreur fichier (format, taille) : message inline sous la zone de dépôt

3.2 EDA — Analyse Exploratoire

Page de visualisation des statistiques calculées sur le dataset. Chaque colonne a sa propre section avec les graphiques adaptés à son type.

Composants à afficher

- En-tête global : dimensions (lignes × colonnes), taux de doublons, types de colonnes
- Carte par colonne avec : type sémantique, % valeurs manquantes, valeurs uniques
- Graphiques selon le type de colonne :
 - Colonnes numériques : histogramme de distribution, box plot, indicateur de skewness
 - Colonnes catégorielles : bar chart des top valeurs, pie chart si ≤ 6 catégories
 - Matrice de corrélation (heatmap) pour les colonnes numériques
 - Graphique de corrélation avec la cible (bar chart horizontal trié)
- Section patterns de valeurs manquantes simultanées si détectées

Composants graphiques recommandés (Recharts)

Type de donnée	Graphique principal	Graphique secondaire
Numérique continue	BarChart (histogramme)	BoxPlot (custom SVG)
Catégorielle	BarChart horizontal	PieChart si ≤ 6 cats
Corrélations	Heatmap (custom)	ScatterPlot sur demande
Cible classification	PieChart classes	BarChart déséquilibre
Valeurs manquantes	BarChart % par colonne	Missing heatmap

3.3 Diagnostic & Preprocessing — Cœur de l'application

Page principale de l'application. Elle combine l'affichage des problèmes détectés et l'interface de dialogue avec le LLM pour les résoudre un par un.

Structure de la page

La page est divisée en deux zones principales :

- **Panneau gauche (30%)** : liste ordonnée des problèmes détectés, chacun avec son niveau de sévérité (● critique / ● moyen / ● faible), la colonne concernée et une description courte.
- **Panneau droit (70%)** : interface de traitement du problème sélectionné. Affiche le contexte statistique, les propositions du LLM et le contrôle de décision.

Flux de traitement d'un problème

1. L'utilisateur clique sur un problème dans le panneau gauche
2. Le système appelle POST /api/recommend/{dataset_id} et affiche un loader
3. Les solutions proposées par le LLM apparaissent sous forme de cartes sélectionnables (2-3 options)
4. L'utilisateur sélectionne une solution et clique sur Appliquer
5. Le système appelle POST /api/execute/{dataset_id} et re-analyse automatiquement
6. Si de nouveaux problèmes apparaissent, ils sont signalés et ajoutés à la file

Composants de la carte solution

- Nom de la solution (ex: Imputation par la médiane)
- Explication du LLM : pourquoi cette solution est adaptée à ce cas précis
- Effets secondaires potentiels signalés clairement
- Badge recommandé sur la solution conseillée par le LLM
- Bouton Demander du code personnalisé si les solutions standard ne conviennent pas

État du progrès

- Barre de progression : X problèmes résolus sur Y détectés
- Badge de statut sur chaque problème dans le panneau gauche : À traiter / En cours / Résolu / Ignoré
- L'utilisateur peut marquer un problème comme Ignoré pour passer au suivant

3.4 Export — Dataset propre

Page finale affichée quand tous les problèmes critiques sont résolus (ou ignorés).

- Résumé des transformations appliquées (liste chronologique)
- Aperçu des 10 premières lignes du dataset transformé (tableau)
- Comparaison avant / après : lignes, colonnes, % de valeurs manquantes
- Bouton de téléchargement du dataset transformé (CSV ou format original)
- Bouton de téléchargement du dataset original brut

4. Stack technique recommandée

Couche	Outil	Rôle
Framework	React + Vite	Interface réactive, build rapide
State global	Zustand	Session UUID + état du dataset courant
Requêtes API	TanStack Query + Axios	Cache, loading states, refetch automatique
UI Composants	shadcn/ui	Composants accessibles, personnalisables
Graphiques	Recharts	Histogrammes, bar charts, pie charts
Routing	React Router v6	Navigation entre les 4 pages
Style	Tailwind CSS	Inclus avec shadcn/ui

5. Architecture des dossiers

```
frontend/src/
├── api/
│   ├── client.js           # Instance Axios (baseUrl + headers session UUID)
│   ├── dataset.js         # upload, getDataset, preview, download
│   ├── eda.js             # runEDA
│   ├── diagnostic.js      # runDiagnostic
│   └── analyze.js         # fullAnalysis (EDA + diagnostic + ordre LLM)
```

```

├── recommend.js      # getRecommendations, generateCode
├── execute.js        # applyTransformation, executeCode
├── components/
│   ├── ui/           # shadcn/ui (Button, Card, Badge, Dialog...)
│   ├── charts/
│   │   ├── HistogramChart.jsx
│   │   ├── BoxPlot.jsx
│   │   ├── BarChart.jsx
│   │   ├── PieChart.jsx
│   │   ├── CorrelationHeatmap.jsx
│   │   └── MissingPatternChart.jsx
│   ├── dataset/
│   │   ├── UploadZone.jsx
│   │   ├── DatasetPreview.jsx
│   │   └── ColumnBadge.jsx
│   ├── eda/
│   │   ├── ColumnCard.jsx
│   │   └── EDAOverview.jsx
│   ├── diagnostic/
│   │   ├── ProblemList.jsx
│   │   ├── ProblemCard.jsx
│   │   └── SeverityBadge.jsx
│   └── preprocessing/
│       ├── SolutionCard.jsx
│       ├── SolutionPicker.jsx
│       ├── CodeViewer.jsx
│       └── ProgressBar.jsx
├── pages/
│   ├── Home.jsx      # Upload + configuration
│   ├── EDA.jsx        # Visualisation statistiques
│   ├── Preprocessing.jsx # Diagnostic + résolution interactive
│   └── Export.jsx     # Résumé + téléchargement
├── store/
│   └── sessionStore.js # Zustand : sessionUUID + datasetId + edaResult +
├── problems
│   ├── hooks/
│   │   ├── useDataset.js
│   │   ├── useAnalysis.js
│   │   └── usePreprocessing.js
│   └── utils/
│       ├── formatters.js # Nombres, pourcentages, dates
│       └── colors.js     # Palettes sévérité, types de colonnes

```

6. État global (Zustand)

Le store Zustand centralise tout l'état de session. Il persiste le UUID en localStorage et maintient les données actives en mémoire.

```

// store/sessionStore.js
const useSessionStore = create((set) => ({

```

```

sessionUUID:    null,           // UUID anonyme persisté
datasetId:      null,           // ID du dataset actif
datasetMeta:    null,           // nom, colonnes, target, task_type
edaResult:      null,           // résultat EDA complet
diagnosticResult: null,         // problèmes détectés
orderedProblems: [],            // problèmes dans l'ordre LLM
problemStatuses: {},            // { problemIndex:
'pending'|'resolved'|'skipped' }
currentProblem: null,           // problème en cours de traitement
history:        [],             // transformations appliquées
});

```

7. Design system

Palette de couleurs

Token	Valeur	Usage
primary	#1E3A5F	Titres, fond header
primary-mid	#2E6DA4	Liens, accents, bordures
accent	#E8A838	Badge recommandé, highlights
critical	#DC2626	Sévérité critique
warning	#D97706	Sévérité moyenne
success	#16A34A	Problème résolu, succès

Typographie

- Police principale : Inter ou Geist (via Google Fonts ou CDN)
- Taille de base : 14px / line-height 1.6
- Titres de page : text-2xl bold
- Labels de sections : text-sm uppercase tracking-wide text-muted

Composants shadcn/ui utilisés

- Card — conteneur de colonne EDA, carte solution, carte problème
- Badge — sévérité du problème, type sémantique de la colonne
- Dialog — confirmation avant application d'une transformation
- Progress — barre de progression du preprocessing
- Tabs — basculer entre les vues d'une même colonne (stats / graphique)
- Skeleton — état de chargement pendant les appels API
- Tooltip — infobulles sur les métriques statistiques

8. Interactions clés à implémenter

Interaction	Déclencheur	Résultat
Upload dataset	Drop fichier ou click	Appel API upload + redirection EDA
Sélectionner un problème	Click sur un item de la liste	Charge les recommandations LLM
Choisir une solution	Click sur une carte	Mise en évidence de la sélection
Appliquer	Click Appliquer	Dialog confirmation → exécution → re-analyse
Ignorer un problème	Click Ignorer	Statut skipped, passage au suivant
Demander du code	Click Code personnalisé	Appel /generate-code → affichage CodeViewer
Exécuter le code LLM	Click Exécuter le code	Appel /execute/code → re-analyse
Télécharger dataset	Click Télécharger	Appel /download → fichier dans navigateur